

关于 AUTOSAR 中 DCM (ISO14229 UDS) 模块的理解

本篇文章主要介绍基于 ISO14229 的 DCM 模块的理解。

阅读本篇文章希望达到的是：

UDS 是干什么的，

ISO14229 是如何定义规则的，

希望接下来的阅读让你不虚此行。

1. UDS 是干什么的？

UDS 全称是 Unified Diagnostic Services，即 统一诊断服务。其最重要的作用就是用来诊断汽车的故障的，当然不仅仅是这个用处，它还可以用来进行汽车的下线检测，比如一般车辆会把 VIN 码写入汽车中的各个零部件中（ECU），比如可以矫正角度，比如可以记录一些和产线相关的信息等等。

那么 UDS 是如何去诊断故障的呢？这里包含两种方式，一种为在线诊断（OBD），一种为离线诊断，前者一般用于传统燃油车中与排放相关的诊断，后者主要是非排放相关的。因为我主要做新能源汽车这一块，因此对非排放相关的诊断理解更多一点，（关于 OBD 可参考 ISO15031）。

那么非排放相关的故障是如何诊断的呢？首先汽车中的每个 ECU 都按照规则存储故障信息，例如 BMS 发生了欠压故障，那么这个时候 BMS 就记录发生故障时刻的 DTC（故障码），以及在故障发生时刻便于查找故障的快照信息或冻结帧信息（例如这个时刻 BMS 的电压、电流等等信息），这些信息是便于查找故障的信息。

为了便于理解，有必要解释一下几个名词：

DTC：诊断故障代码，其意思就是通过一个代码 代表一个故障；

快照/冻结帧：指发生故障时刻的一些便于排查故障的信息；

扩展信息：这个是指除快照之外，与故障相关的一些信息，例如故障的发生次数、老化次数等等。

以上讲了 ECU 是如何记录故障信息的，下一步讲我们如何去诊断我们的汽车发生了什么故障，我们接着以 BMS 发生了欠压故障导致车辆无法行驶为例，那么故障车一般会被拖到 4s 店进行维修，4s 店为快速定位车发生了什么故障，这个时候他们会使用诊断仪，一键广播查找车上所有零部件上发生的故障信息，这样可以很快知道是 BMS 发生了故障，并且发生的故障是欠压故障，那么欠压故障是因为什么导致的呢，这个时候就要分析快照数据了，根据快照数据，快速的找到可能是因为电池包本身的电压过低导致。以上就讲完了 UDS 在车上是如何使用的，也就是 UDS 是干什么的问题。

读完这一段，你会想，UDS 确实有点用处，因为你要想，车上的零部件都是成百上千的，要快速精确定位故障不是一件容易的事情，我希望你有兴趣继续读下去，下面我们就要讲讲 UDS 能有这种神奇功能，他是如何去规定的。

2.ISO14229 是如何定义规则的？

要快速定位车上的故障，那么就要求车上所有的 ECU 都遵守相同的规范，同时全世界又有各种各样的主机厂，假设每个厂商都自己去定义规范，可想而知，到最后几千万上亿的车到最后可能需要各种各样的 4s 店。因此 ISO 就出台定义了一整套 UDS 相关的规范，这样大家都遵守一定的规范办事，也就便于后面的维护管理了（以上闲扯几句，可跳过）。

首先，我们大致推断一下 ISO 想干一件什么事，也就是这个规范它应对的需求是什么。

从 1 中提到的 UDS 要干的什么事，我们大致了解，这套规则需要所有的汽车厂商遵守一套协议，同时能满足所有厂商下线检测以及诊断的功能。

首先我们讲讲一般汽车厂商下线检测需要检测些什么东西，首先主机厂肯定需要的一个信息就是为生产的每一辆车分配一个标识码(VIN 码，有兴趣的可百度一下)，这个 VIN 码可以唯一标识这辆车，下线检测的时候，就可以通过 UDS 写入到汽车的 ECU 中，所以这就会用到要写入信息的功能，但是，既然是唯一的，因此不能随便写入啊，所以就要分配权限，只能通过安全认证的人才能写入。写入之后，当然就要读出来，要不怎么知道车里面的 VIN 码呢。以上就引出了 UDS 的三个功能，读数据、写数据、以及权限检查。

当然，下线检测当然不仅仅是这些功能，例如需要调整电机的零位角度或者 ECU 自检等，这个时候就需要 UDS 支持复杂的功能，UDS 管这种支持复杂功能的服务叫做例程，你可以把例程理解为一段比较复杂的逻辑程序。

有时候，我们需要去控制电路上的引脚，来调试或者检测制定的功能，例如实现一个点灯的操作，这个时候，UDS 就提出了 IO 控制。

假如，我们想要复位一些车上的 ECU，我们当然不希望还要通过拧钥匙这种操作来完成，因此 UDS 中提供了各种各样的复位操作；

假如，我们发生车上的某个 ECU 中的程序有个 BUG 导致车坏了，要升级怎么办，我们当然不希望拆掉一个 ECU 换一个，那么 UDS 就支持烧写程序的操作；

那么，还有一个需求就是，假如车上某个部件坏了，那么会有故障信息存储下来，这个时候为了排查故障，我们需要快速找到坏的部件，然后为了快速定位故障，我们需要知道故障发生时刻的车的状态，等车修好了，我们不需要这个存储的故障信息了，我们一般需要删掉这个故障信息，以腾出空间为后面发生的故障存储。也就是我们需要读取故障信息，以及清除故障信息的操作。

以上讲的是常用的 UDS 的一些操作，当然不仅仅是这些，比如烧录的时候，为了降低总线线路传输数据的负载率，一般需要控制车上各个 ECU 收发信息的开关，同时刚烧录的时候，一般不能开启故障存储，因为这个时候地址里面的数据不是真实发生的故障。因为 UDS 一般都是涉及嵌入式的软件，为了更自由的读写数据，还可以通过读写地址，当然也支持读取多个数据（规定上限 4095 个），因为 UDS 采用的是一问一答的形式，假如我们要动态实时的知道某个值怎么办，这个时候 UDS 当然支持周期读取数据的操作等等类似的内容。

希望你能耐心读取一下以上的需求，因为对理解 14229 肯定有点帮助的。

从上面对于应用场景的大致需求，我们下面一一对号入座来谈谈 14229 是如何定义规则。

10 服务：

首先需要隆重介绍的一个服务是 10 服务，这个服务用来做什么？打个比方来说，假如有三个池塘，A 池塘用来放草鱼和鲶鱼，B 池塘用来放观赏性的锦鲤，C 池塘用来放小型鲨鱼。那么我们 10 服务中的每个状态对应一个池塘，而每个池塘里的鱼就是 14229 里面对应的所有服务（所谓服务就是一套数据定义规则），里面 A 池塘对应 10 服务的默认状态，里面的草鱼和鲶鱼，对应在这种默认会话状态下支持的服务，同样 B 池塘对应 10 服务的编程模式，里面的各种锦鲤对应编程模式下支持的各种服务。C 池塘对应扩展模式，小型鲨鱼对应在该模式下支持的服务。是不是还是有点不理解，我再简单说一下，也就是说 10 服务用来切换三种或更多模式，默认模式用来支持默认状态下支持的各个服务，编程模式用来支持与编程相关的服务，扩展模式一般用来在非默认模式下支持的一些服务。

为什么要这么做？其实就是为了管理各个服务，让各个服务在一个池子里面，只有进入这个池子，你才有限去访问各个服务。

数据应答规则举例大致说明一下：

请求: CANID 02 10 03 （02 对应数据长度，10 表示 10 服务，03 表示 10 服务的子服务，该请求的目的就是切换到扩展模式，其他模式切换类似）

答复: CANID 06 50 03 xx xx xx xx （06 对应数据长度，50 表示 10 服务（+0x40），03 表示 10 服务的子服务，该答复的目的就是切换到扩展模式，其他模式切换类似）

有兴趣的朋友可以深入研究一下。

11 服务：

对应的就是复位服务，其中包括了子服务硬件复位 01 子服务 KeyOfOn 复位 02 子服务软件服务 03；其中三种子服务都是通过操作软件来进行对应的复位操作。

数据应答规则

请求: CANID 02 11 01

答复：CANID 02 52 01

14 服务：

对应的就是清除故障服务

他可清除一个故障信息，也可以清除一组故障信息。

应答规则后面不讲了，有兴趣的朋友可支持参考 14229 协议。

19 服务：

这个服务主要用来，查询故障信息，这个服务非常复杂，需要费很大的力气才能讲清楚，下面我仅仅对常用的几种子服务进行介绍，其实所有定义的服务，无非就是为了快速查找到需要的故障信息。

01 子服务：通过状态掩码的形式去，查找该状态掩码匹配的故障个数，其实意思就是查找故障个数，这里稍微解释一下状态掩码，在 UDS 规定里面，每个故障都对应 8 个状态，每个状态对应一个 bit，比如状态 0 对应 bit0，表示当前发生的故障，状态 3 对应 bit3，表示已经确认的故障等等，那么状态掩码的意思就是用一个 8bit 的数去按位与，如果与的结果与的结果非 0 表示匹配到了，然后故障数就++。

02 子服务：通过状态掩码的形式去，查找匹配的故障，以及故障的状态。

04 子服务：请求指定故障码（DTC）的快照信息，意思就是说为了找到故障的原因，查找故障发生时刻的一些数据，来分析故障原因。

06 子服务：请求指定故障码（DTC）的扩展信息，就是想了解一些该故障的一些其他信息，比如发生的次数、自恢复的次数等，具体数据可自定义。

0A 子服务：通过该服务可获取所有支持的故障码和故障状态信息，注意是所有的故障及故障码。

这个服务大致简单的讲到这里吧，其实这个服务里面有 N 多内容，需要细细挖掘掌握。

22 服务：

通过数据标识符的形式读取数据，例如我想读电机控制器里面的电机转速的信息，我就定义电机转速这个变量 标识符为 0x2000，（这是因为总线上基本不搞变量啊），所以我去请求电机转速时，我只需读该标识符的数据即可。

2E 服务：

与 22 服务对应，表示通过数据标识符写入数据，一般来说，一个标识符可支持读，写操作是根据需求可选的。

23/3D 服务：

23 是通过地址读数据

3D 是通过地址写入数据，一般用的较少。

24/2A/2C/86 服务：

这类服务用的较少，这里不再赘述：

27 服务：

这个服务就是用来权限管理的，他的权限进入方式是：

诊断仪		电机控制器 ECU
1. 请求种子	-->	计算种子
2. 接收种子	<--	答复种子
3. 发送密钥	-->	计算密钥 对比密钥
4. 获取权限	<--	返回权限切换状态

比如你要向 ECU 写入数据，一般人当然不能随便写，只有专门懂的人才能写入，因此需要权限来保证。

28 服务：

通信控制，包括对发送和接收消息的开关控制。

31 服务：

例程控制，比如一些复杂的操作需要用他来实现，目前比较通用的包括烧录时的数据完整性检查、擦除内存等等。

2F 服务：

IO 控制，主要用于对一些输入输出口的调试控制，目前这一块接触较少，比如一些传感器的开关控制。

34/35/36/37 服务：

和数据传输有关的服务，包括请求传输、请求下载、数据传输、数据上传、退出传输等，这些服务和 BootLoader 相关。

85 服务：

用于控制故障的更新，包括开启和关闭故障更新，比如关闭之后不允许故障、故障状态、故障记录等信息的更新。开启则反之。

以上服务讲的比较浅，有兴趣的朋友可以对比着和 14229 看一下，相信会对理解有一定帮助。